

Supplementary Material

LoHR-Bench: A Dual-Level Benchmark for Extended Long-Horizon Robot Manipulation

S1. BENCHMARK TASKS

Table S1 summarizes the 20 tasks in LoHR-Full across four suites. An asterisk marks the eight LoHR-Core tasks, with two tasks from each suite. Each entry gives a task description, symbolic plan length (# Sym.), and average episode length in control steps. Figure S3 shows the task settings.

S2. TASK SPECIFICATION

Each task combines a PDDL problem, a PDDL domain, and a spatial configuration. The problem specifies objects, initial relations, and goals. The domain defines action preconditions and effects. The spatial configuration maps named positions to scene coordinates.

Listing S1 shows the objects, initial stacking relations, and goal for reverse stacking. The goal specifies the reversed tower at the target position. The listed goal uses a representative ordering of the gray cubes. Evaluation allows a consistent permutation within the interchangeable group declared in the initial state.

Listing S1: Problem specification for reverse stacking (excerpt).

```
(:objects
  panda_wristcam - robot
  cubeA cubeB cubeC cubeD cubeE
  cubeF cubeG cubeH cubeI cubeJ - block
  pos11 - position)

(:init
  (gripper-empty panda_wristcam)
  (clear cubeJ)

  (on-object cubeB cubeA) (on-object cubeC cubeB)
  (on-object cubeD cubeC) (on-object cubeE cubeD)
  (on-object cubeF cubeE) (on-object cubeG cubeF)
  (on-object cubeH cubeG) (on-object cubeI cubeH)
  (on-object cubeJ cubeI)

  (interchangeable cubeF cubeG)
  (interchangeable cubeF cubeH)
  (interchangeable cubeF cubeI)
  (interchangeable cubeF cubeJ)
  ;; Other initial facts omitted.
)

(:goal
  (and
    (object-at cubeJ pos11)
    (on-object cubeI cubeJ) (on-object cubeH cubeI)
    (on-object cubeG cubeH) (on-object cubeF cubeG)
    (on-object cubeE cubeF) (on-object cubeD cubeE)
    (on-object cubeC cubeD) (on-object cubeB cubeC)
    (on-object cubeA cubeB)))
```

In reverse stacking, the robot removes blocks from an existing tower and rebuilds it in reverse order. Listing S2 illustrates the symbolic transition for unstacking. The top block must be clear, and the robot's gripper must be empty. After the action, the robot holds the removed block, its

previous support becomes clear, and the original stacking relation no longer holds.

Listing S2: Simplified task-level unstack action for reverse stacking. Our TAMP baselines adapt standard planner implementations to LoHRbench with task-specific planning domains and geometric feasibility constraints.

```
(:action unstack
  :parameters (?r - robot ?top ?bottom - block)
  :precondition (and
    (on-object ?top ?bottom)
    (clear ?top)
    (gripper-empty ?r))
  :effect (and
    (holding ?r ?top)
    (clear ?bottom)
    (not (on-object ?top ?bottom))
    (not (clear ?top))
    (not (gripper-empty ?r))))
```

The spatial configuration maps the symbolic target position pos11 to coordinates in the simulated scene.

```
{
  "positions": {
    "pos11": {
      "region": "table-on",
      "fixed_coords": [0.0, 0.2, 0.02]
    }
  }
}
```

Complete task configurations are provided under benchmark/table-top/<category>/<task>/config/ in the repository.

S3. INTERFACES FOR LONG-HORIZON REASONING

LoHRbench provides a unified interface for long-horizon reasoning. It exposes task and state context, multiple subgoal representations, and execution feedback. Together, these interfaces allow a method to interpret the task, reason about the evolving scene, formulate intermediate goals, and update subsequent decisions based on execution outcomes.

We organize the interface into three parts that follow the reasoning and execution process. Part A provides the context for selecting the next action or subgoal. Part B represents the intended outcome of the next stage. Part C connects the selected subgoal to execution and returns feedback for subsequent decisions. Methods can combine these interfaces according to their own planning and control structure.

A. Task and State Context

Part A exposes the task and state interfaces listed in Table S2.

These interfaces allow a method to construct the task representation from the PDDL domain and problem, reason from the observations available in its evaluation setting, and track decisions against the recorded execution trace. The

TABLE S1: LoHR-Full tasks across four suites. An asterisk marks the eight LoHR-Core tasks.

Task	Task description	# Sym.	Avg. steps
Long-horizon			
stack_cube_10*	Stack 10 cubes together, starting with the red cube.	20	2207
sort_and_store	Sort tabletop objects into two boxes: toys and balls in one box, tools in the other.	20	2454
reverse_stack*	Reverse a stack of 10 cubes.	20	2207
stack_cube_pyramid_simple	Build a 10-cube pyramid with four layers containing 4, 3, 2, and 1 cubes, respectively.	20	2172
stack_cube_hanoi	Solve a Tower-of-Hanoi-style rearrangement with colored cubes using pick-and-place under Hanoi constraints.	22	2127
Active exploration			
stack_active*	Retrieve cubes from a closed drawer and stack them together, starting with the red cube.	10	2092
pick_active*	Retrieve the can, screwdriver, and cup from a two-layer drawer and place them at their target locations.	10	2134
drawer_search_hard	Retrieve the can, screwdriver, and cup from a cluttered two-drawer cabinet and place them at their target locations.	10	2127
stack_cube_cabinet	Retrieve cubes from a closed two-level cabinet and stack them.	9	1378
food_retrieval	Retrieve two strawberries and two apples from the cabinet. Place the strawberries in the bowl and the apples on the plate. Both containers are on the table.	9	2024
Tool use			
fruit_placement*	Move four strawberries to the target position. A bowl is available to help transport them.	10	1349
repackage*	Put all cubes into the bowl and stack the bowl on the plate.	6	634
container_transfer	Transfer items from one container into another and place the destination container on a plate.	5	1731
food_storage_simple	Store a bowl containing a strawberry inside a locker.	11	1939
push_T_items	Move the cubes and strawberries to the target location. Several T-shaped blocks of different sizes are available to help move them.	10	1541
Cluttered manipulation			
stack_clutter*	Stack three cubes together in a compact environment with surrounding obstacles.	18	1516
clutter_packing*	Pick three cubes from a cluttered tabletop and place them into a bowl.	18	1615
tall_clutter	Retrieve a cube surrounded by taller cubes and place it into a bowl.	14	1175
clutter_draw	Extract the red cuboid from beneath the middle column of the three-column stack.	14	1620
clutter_push_cube	Push a cube to a target location through clutter, moving obstacles that block the path.	9	983

privileged snapshot is available only in settings that permit simulator state. Visual methods instead reason from obs and their own state estimates.

Relations are represented as grounded predicates, such as (on-object cubeI cubeJ), which states that cubeI is on cubeJ. Each relation has a status of supported, contradicted, or unresolved. Each state field also records whether it comes from simulator state or a perception module. Expected action effects are recorded separately from observed outcomes.

B. Subgoal Representations

Part B defines the four SubgoalSpec representations listed in Table S2. The subgoal specifies the intended outcome, while the choice of executor remains separate and is handled through Part C.

Each intermediate goal is a SubgoalSpec with

fields subgoal_id, representation, payload, completion, and monitored_relations. The payload fields depend on the selected representation.

The four representations connect the same task stage to different classes of methods. Symbolic predicates support task planning and predicate-based verification. Language and visual targets can condition learned policies or grounding models. Spatial targets connect an object or end-effector goal to motion planning and geometric control. Their shared SubgoalSpec fields keep the target and its completion conditions aligned across representations. A registered adapter maps the selected representation to the input expected by a planner, policy, or verifier. A method can support one or several representations.

For example, the reverse-stacking goal of placing cubeI on cubeJ can be expressed as the symbolic predicate (on-object cubeI cubeJ), the language goal “Place

TABLE S2: LoHRbench interfaces for long-horizon reasoning, organized into task and state context, subgoal representations, and execution and feedback.

Interface	Definition	Inputs and outputs
Task and State Context		
Task specification	TAMPEnvironment	domain, problem, position_file, and get_language_instruction()
Observation	TAMPEnvironment	obs structured by observation_space
Symbolic state	get_symbolic_state()	Grounded predicate tuples
Privileged snapshot	current_state_snapshot	symbolic_state, object_poses, articulation_states, and robot_tcp_pose
Execution trace	pddl_action_ranges	pddl_action, start and end steps, step count, and success
Subgoal Representations		
Symbolic	SubgoalSpec (symbolic)	predicates, object_bindings
Language	SubgoalSpec (language)	text, object_bindings
Visual	SubgoalSpec (visual)	image, camera, object_regions
Spatial	SubgoalSpec (spatial)	subject, reference_frame, target_pose or target_region, and tolerance
Execution and Feedback		
Execution	TAMPEnvironment.step()	action_name, param_mapping, and/or action_seq
Predicate check	TaskPlanEnv.check_predicates()	Grounded preconditions or effects and whether they hold
Feedback	Return from TAMPEnvironment.step()	obs, reward, done, truncated, and info

cubeI on cubeJ,” a visual target showing the completed placement, or a spatial target defined relative to cubeJ. These representations can share the same completion check, which verifies the stacking relation after the objects are released and settled.

C. Execution and Feedback

Part C uses the execution and feedback interfaces listed in Table S2. It separates action execution from subgoal verification because an executor can finish without achieving the intended outcome.

These interfaces connect a selected subgoal to physical execution. High-level methods submit a symbolic action and its object bindings. Low-level methods submit an action sequence, while bilevel methods provide both. Predicate checks and the returned `info` fields distinguish action execution, task progress, goal completion, and budget exhaustion. This feedback can be used to select the next subgoal or revise the current plan.

Execution status records whether the requested operation finished, while subgoal status records whether its intended outcome was verified. Completion checks evaluate the outcome after execution. Relation monitors track conditions that should hold during execution while allowing motion required by the task. Placement completion uses the released and settled conditions defined in Section S4.

A verification result can be supported, contradicted, or unresolved. A timeout report retains the available execution results and consumed budget. Each report is added to the execution history in Part A, closing the loop for the next planning or policy decision. The Agentic Hybrid in Section S6-C illustrates one use of these interfaces.

S4. PREDICATE EVALUATION

LoHRbench evaluates goal relations in the simulated scene. An object registry maps PDDL symbols to robots, objects, and

TABLE S3: Logical predicates and their evaluation criteria.

Predicate	Meaning and evaluation
robot-at	The robot base is within the positional tolerance of the specified location.
object-at	The object is within the positional tolerance of the specified location.
holding	The robot is grasping the specified object.
gripper-empty	The robot is not grasping any registered object.
clear	The object is released and its top is unoccupied, or the specified placement location is unoccupied.
on-object	The relative height and horizontal alignment satisfy the stacking relation.
in-object	The object lies within the container’s geometric containment region.
in-drawer	The object is assigned to the specified drawer in the initial task state.
accessible	The object’s access condition is satisfied, such as its containing drawer being open.

named positions. Spatial placement checks require the relevant objects to be released and settled. Table S3 summarizes the predicates.

Figure S1 illustrates partial goal satisfaction in 10-cube stacking. We use the evaluation metrics defined in Section III-F of the main paper.

S5. EXTENDING LOHRBENCH

A. Adding New Tasks

Adding a new task is relatively straightforward and follows a well-defined, template-based process. A task is specified by creating a directory under `benchmark/table-top/{task_type}/{task_name}/`. Its `config/` subdirectory contains PDDL domain and problem files defining types, predicates, actions, the initial state, and success goals, along with a `position.json` file specifying spatial regions and object placements. Reference symbolic plans are stored in

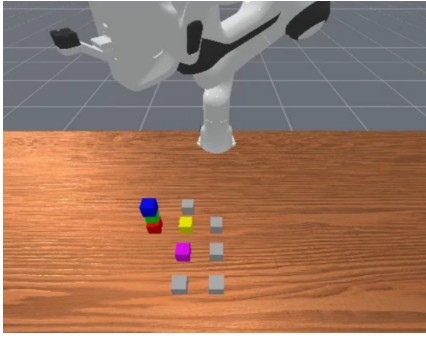


Fig. S1: Partial goal satisfaction in 10-cube stacking. The illustrated state satisfies two of the nine target stacking relations, giving a fraction of $2/9$.

the `solution/` subdirectory. If existing action primitives (e.g., pick, place, move-to) support the required behaviors, a compatible executor can be reused. Otherwise, a new task executor can be implemented in `src/lohrbench/motion_planner/tasks/tabletop/` to translate symbolic actions into motion commands. Finally, for symbolic action execution, the task is mapped to its executor by adding an entry to `TASK_TO_EXECUTOR` in `src/lohrbench/env/utils.py`.

B. Integrating Other TAMP Methods

The PDDLStream integration provides a reference for incorporating other TAMP methods into LoHRbench. A method-specific adapter translates the benchmark task specification and current observations or state estimates into the planner’s internal representation, then maps its output to executable actions. LoHRbench exposes interfaces for submitting symbolic actions with object bindings, continuous control sequences, or both. This allows a TAMP method to retain its own task and motion planning components while using the benchmark environment and evaluation functions. Compatible task executors can be reused for symbolic action execution. For methods that reuse these executors, alternative motion-planning backends can be connected through `BaseMotionPlanner`, with `CuRobo` and `ManiSkill/MPLib` as existing examples.

C. Supporting Other Simulators

LoHRbench currently uses `ManiSkill 3` and `SAPIEN` for simulation. Its PDDL domain and problem files describe symbolic task structure, including action preconditions, effects, and goals, separately from simulator execution. Porting the benchmark to another simulator would require adapting scene and asset loading, robot control, and predicate evaluation to the target simulator’s APIs. Spatial configurations would also need to be mapped to the target simulator’s coordinate and asset conventions. The adapted predicate evaluators should preserve the task relations and satisfaction criteria defined in Section S4, allowing the same symbolic task definitions and evaluation metrics to be reused.

TABLE S4: Policy observation and action settings. Joint denotes seven joint position targets and a gripper command. EEf delta denotes a 6D end-effector pose delta and a gripper command. Obs., Pred., and Exec. denote observation, prediction, and execution horizons, respectively, measured in steps.

Method	Proprio.	Action	Obs.	Pred./Exec.
ACT	Yes	Joint	1	30/8
DP	Yes	Joint	2	16/8
OpenVLA-oft	No	EEf delta	1	8/8
π_0 (FAST)	Yes	Joint	1	16/16

D. Supporting Multiple Robot Configurations

LoHRbench supports extension to additional robot configurations through a central robot registry. Each configuration specifies the simulator robot identifier, arm joint names, state and action dimensions, gripper parameters, and the `CuRobo` configuration. To integrate a robot such as a UR10e or xArm, users register these settings and provide compatible simulator controllers and motion-planning configurations. Task executors and learned-policy interfaces are then adapted to the robot’s kinematics, gripper behavior, and observation and action spaces. The robot object in the PDDL problem selects the corresponding registered configuration.

S6. BASELINE DETAILS

A. End-to-End Learning Baselines

1) *Observation and Action Settings*: We evaluate four end-to-end policies: Action Chunking with Transformers (ACT), Diffusion Policy (DP), OpenVLA-oft, and π_0 (FAST). All policies receive task instructions and RGB images from base and wrist cameras. Table S4 summarizes their proprioceptive inputs, action representations, and observation, prediction, and execution horizons.

2) *Training Configuration*: We train ACT and Diffusion Policy on demonstrations from the eight LoHR-Core tasks, using fixed CLIP text embeddings for language conditioning. We fine-tune OpenVLA-oft and π_0 (FAST) on the same tasks using low-rank adaptation (LoRA).

3) *Action Chunking with Transformers (ACT)*: ACT predicts action chunks in parallel using a conditional variational autoencoder with a transformer decoder. We condition the policy on visual features, robot state, and fixed CLIP text embeddings.

4) *Diffusion Policy (DP)*: Diffusion Policy models a conditional distribution over action sequences with a temporal U-Net. It is trained to predict injected noise from visual observations, robot state, and fixed CLIP text embeddings, and generates actions through iterative denoising at inference.

5) *OpenVLA-oft*: OpenVLA-oft predicts continuous action chunks from visual observations and task instructions using an L1 regression action head. Following the end-effector action representation used in OpenVLA pretraining, we represent each action as a 6D end-effector pose delta and a gripper

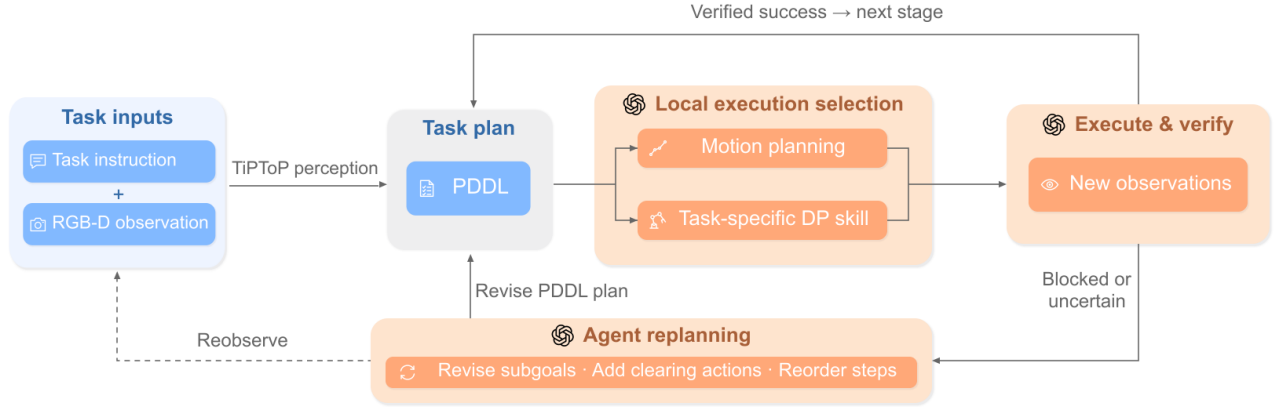


Fig. S2: Agentic Hybrid framework. Astra selects between motion planning and learned primitives and uses updated observations and execution feedback to revise intermediate goals and action order.

command. We adapt the pretrained model to LoHR-Core using LoRA.

6) π_0 (FAST): π_0 (FAST) generates tokenized action chunks autoregressively from visual observations, robot state, and task instructions. We adapt the pretrained model to LoHR-Core using LoRA.

B. Task and Motion Planning Baselines

1) *PDDLStream*: PDDLStream receives the benchmark PDDL goal and simulator state. Continuous streams sample grasps and placements and invoke inverse kinematics and CuRobo motion planning to produce executable actions. We use the adaptive PDDLStream algorithm with closed-loop execution. The planner reconstructs the scene state from the simulator when replanning is needed.

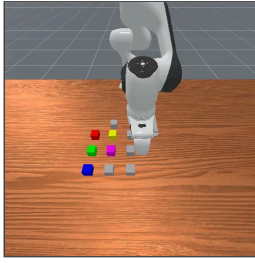
2) *TiPToP*: TiPToP constructs a planning problem from the task instruction and camera observations. FoundationStereo estimates depth from stereo RGB images. Gemini identifies task objects and translates the instruction into a symbolic goal, while SAM2 segments the objects and M2T2 proposes grasps. The resulting scene representation is passed to cuTAMP for task and motion planning, followed by CuRobo trajectory refinement. We evaluate open-loop execution: TiPToP observes the scene initially, refines a complete plan, and executes it.

C. Agentic Hybrid

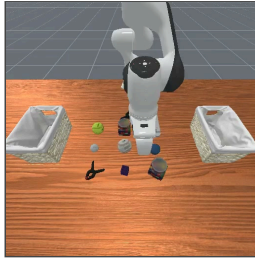
The Agentic Hybrid design illustrates how the reasoning interfaces in Section S3 can support subgoal selection, execution, and plan revision. Figure S2 summarizes the Agentic Hybrid framework. Astra serves two roles in the Agentic Hybrid. First, it selects between a learned primitive and motion planning for each subgoal based on the current observation and skill entry conditions. When needed, MP prepares the arm for learned primitive execution. Second, Astra uses observation and execution feedback to revise intermediate goals and action order when progress is blocked. These revisions aim to avoid spending the remaining planning budget on repeated attempts at the same blocked subproblem.

Each selected operation is formulated as a local planning problem for cuTAMP to solve.

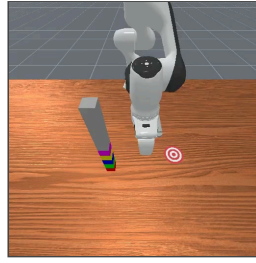
Stage verification checks whether the relations required by the current subgoal hold after execution and whether previously established relations remain valid. Visual and geometric evidence supports three outcomes for each checked relation: supported, contradicted, or unresolved. Astra uses this evidence to decide whether to advance, request another observation, or replan. Benchmark success and progress are evaluated separately using the simulator-based predicates defined in Section S4.



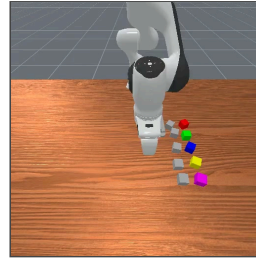
stack_cube_10
Long-horizon



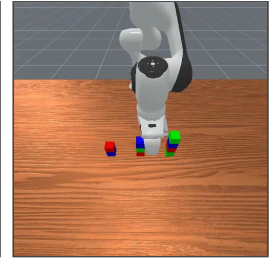
sort_and_store
Long-horizon



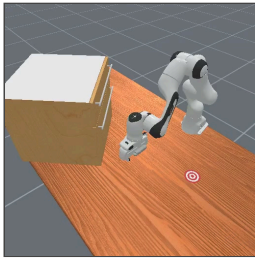
reverse_stack
Long-horizon



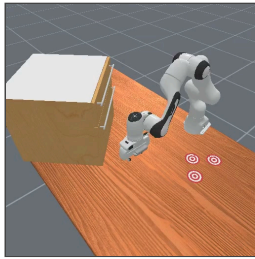
stack_cube_pyramid_simple
Long-horizon



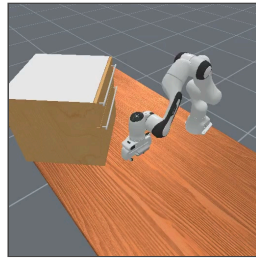
stack_cube_hanoi
Long-horizon



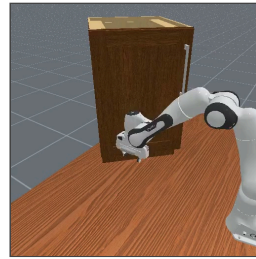
stack_active
Active exploration



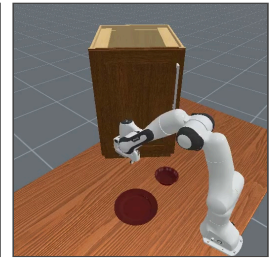
pick_active
Active exploration



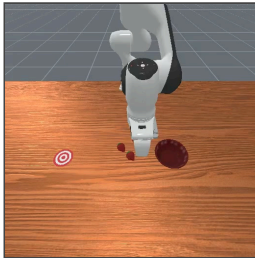
drawer_search_hard
Active exploration



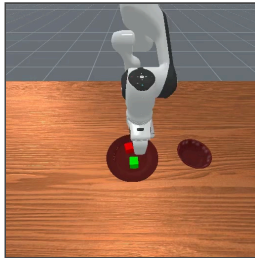
stack_cube_cabinet
Active exploration



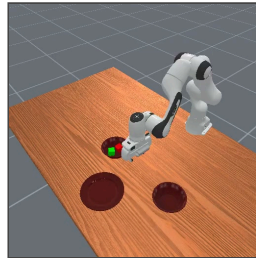
food_retrieval
Active exploration



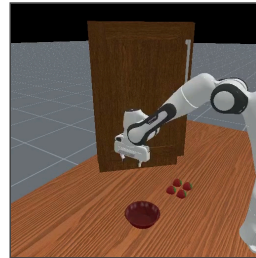
fruit_placement
Tool use



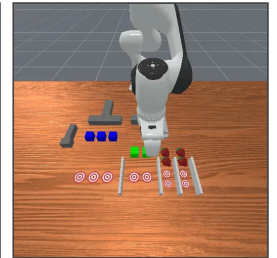
repackage
Tool use



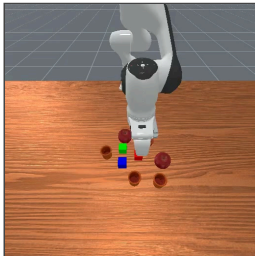
container_transfer
Tool use



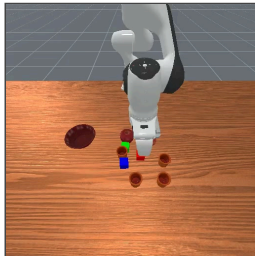
food_storage_simple
Tool use



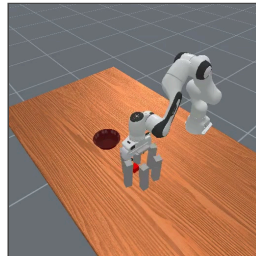
push_T_items
Tool use



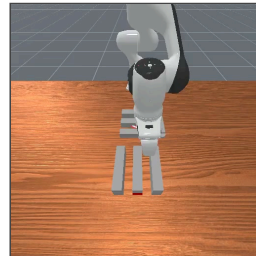
stack_clutter
Cluttered manipulation



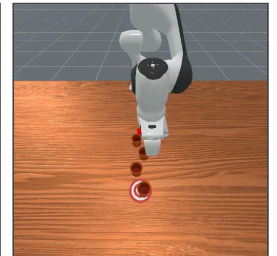
clutter_packing
Cluttered manipulation



tall_clutter
Cluttered manipulation



clutter_draw
Cluttered manipulation



clutter_push_cube
Cluttered manipulation

Fig. S3: Overview of LoHRbench tasks corresponding to Table S1.